Mitchell Bialecki
CSE 536
Project 2/3
11/13/2026

**Introduction**

Fearless conspiracy is a test based social deduction game that I wrote in Rust inspired by the Town of Salem. It supports 3-9 players currently. In the game, Town players try to identify and eliminate the Mafia, while the Mafia players try to eliminate Town, and the Jester tries to get publicly executed. Each player runs a separate client under a different linux account. The communications library is contained in src/comm.rs. Its main API includes ServerTransport::bind, ClientTransport::connect, send, recv_timeout, send_to, send_group, and broadcast, along with close and shutdown_gracefully. For this project, I kept the original socket transport and added Linux isolation to protect private activity and limit resource abuse.

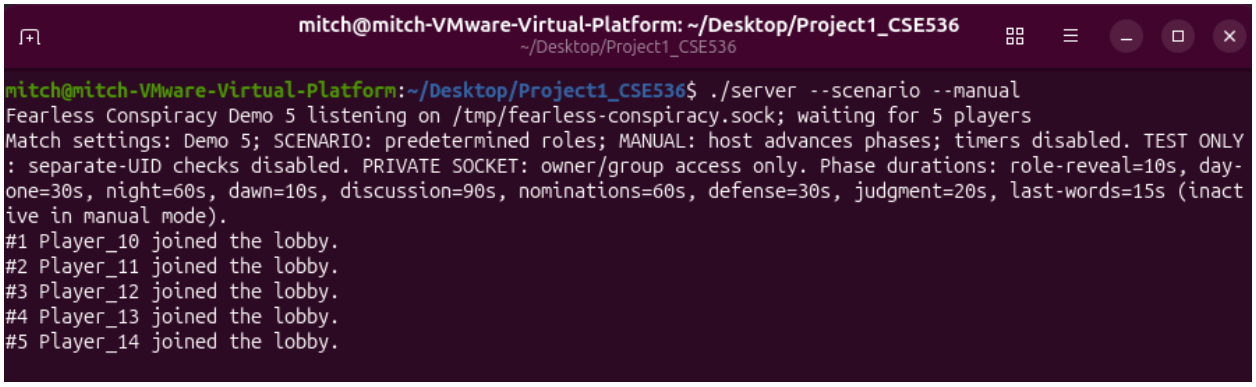
**Original Communications Primitive**

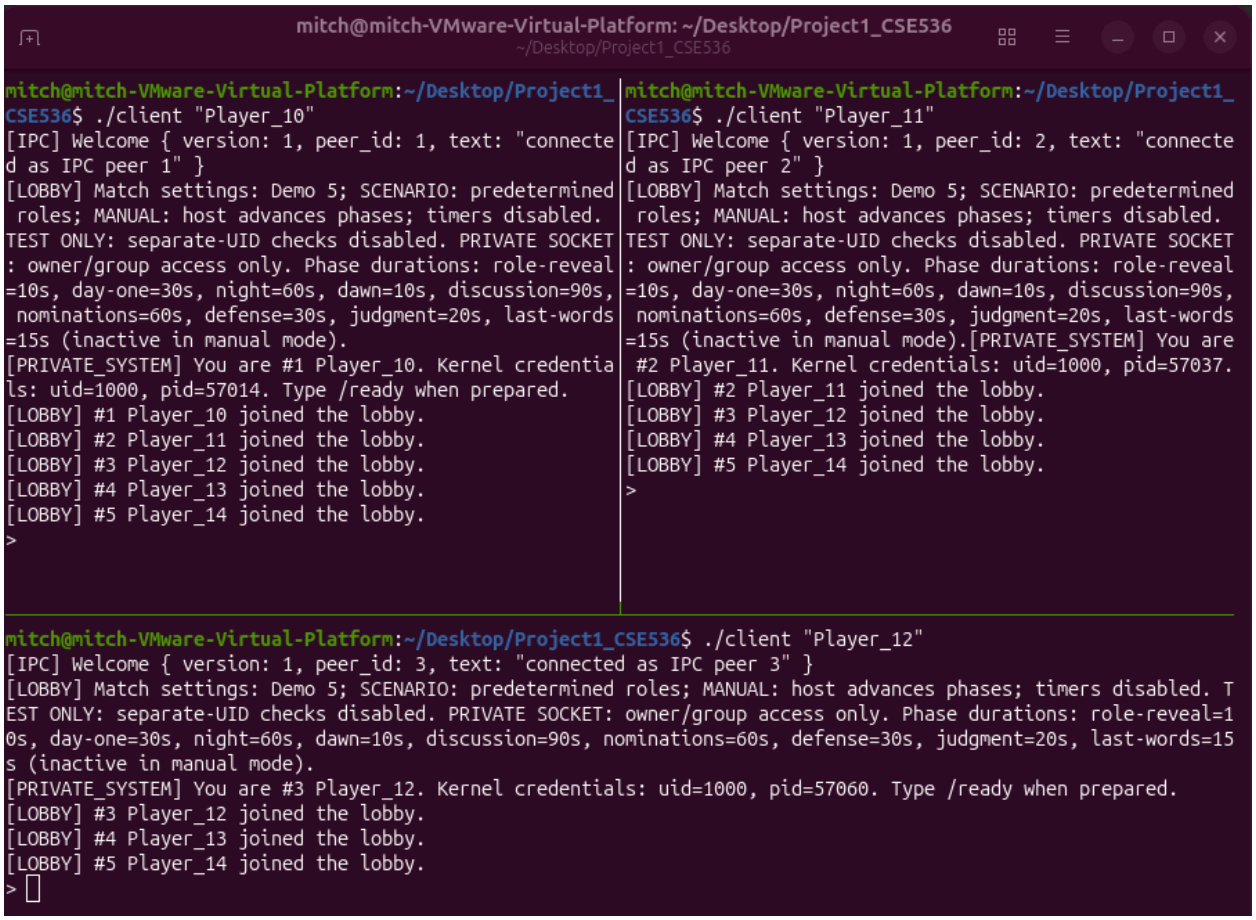
The game uses Linux Unix-domain stream sockets to connect each client process to the server. Because sockets carry a byte stream rather than separate messages, the library adds a four-byte length before each JSON message and handles partial reads and writes. Reader and writer threads allow multiple players to communicate concurrently while the server updates game state sequentially. Bounded queues, size limits, and timeouts help handle malformed or slow connections. Linux peer credentials identify client UIDs. Mafia group messages, and public broadcasts are delivered through the appropriate client connections. Multicast and broadcast are implemented by sending to selected connections rather than by native socket multicast.

**New Isolation mechanisms:**

In the previous project, an issue came up similar to the initial werewolves where other threads could be watched which created a side channel attack where if you knew this was a time where mafia was talking and you see a thread receiving messages, you could deduce they were mafia. To fix this, I created a “protected” launcher that applied hidepid=2 so that separate player accounts are unable to read the server metadata. It also restricts access to other players cgroup statistics which is another thinking that I added. Now, each player receives a systemd user slice with a 25% quota of one CPU, a 192 MiB memory soft limit with a 256 MiB hard limit, no swap, and a 64 task limit. The transport admits only registered player UIDs whose clients belong to the limited slice.

“Protected” mode is now the default. In this mode, the launcher creates the private player accounts, checks permissions and the kernels limits, and starts the game under a separate account without root credentials or capabilities. It saves the previous procfs settings, resource overrides, and cgroup permissions before changing them. This is important so that on shutdown when it stops the game, it removes the automatically managed player accounts and restores the previous settings. If an account was passed in that was already existing, it gets preserved. If for some reason the cleanup process gets interrupted, there is a recovery record so the next launch will finish cleanup.




Game creation and players joining from player and server POV.

```
Ready update: #1 Player_10 is READY (1/5 connected ready).
Ready update: #2 Player_11 is READY (2/5 connected ready).
Ready update: #3 Player_12 is READY (3/5 connected ready).
Ready update: #4 Player_13 is READY (4/5 connected ready).
Ready update: #5 Player_14 is READY (5/5 connected ready).
Game started. Player roles:
  #1 Player_10: Sheriff
  #2 Player_11: Doctor
  #3 Player_12: Lookout
  #4 Player_13: Mafioso
  #5 Player_14: Jester
Current scene: Role Reveal (Day 1)
```

```
> /ready
[PUBLIC_SYSTEM] #3 is ready.
[OK] Ready status confirmed: ready.
[PUBLIC_SYSTEM] #4 is ready.
[PUBLIC_SYSTEM] #5 is ready.
[PRIVATE] You are the Lookout (Town). To learn how your role works and see its commands, enter: /help Lookout
>
```

The game officially starting and giving out roles from both player and server POV.

```
[PRIVATE] You are the Lookout (Town). To learn how your role works and see its commands, enter: /help Lookout
> /help Lookout
[PRIVATE_SYSTEM] Lookout — Town Investigative
Objective: Track visits and compare them with player claims.
Ability: Watch one other living player at night and learn up to three players who visited them.
Commands: /watch ID, /cancel
Target IDs: run /players to see every player's ID, then use that number in commands such as /watch ID or /inves
tigate ID.
Common commands: /role, /players, /say MESSAGE, /whisper ID MESSAGE, /will set TEXT, /will show, /help, /quit
>
```

Help command which helps gives players hints on how to use their role.

```
Current scene: Role Reveal (Day 1)
advance
Scene: Role Reveal (Day 1) -> Day 1 Introductions (Day 1)
```

Server running advance command to skip to next scene

```
afia roster: #4 Player_13 (Mafioso).
[PUBLIC_SYSTEM] [phase 2 · DayOne] Day 1 introductions h
ave begun. No trial is held today.
> /say Hello Everyone! My name is Player_13!
[PUBLIC] #4 Player_13: Hello Everyone! My name is Player
_13!
[OK] Message delivered.
[PUBLIC] #5 Player_14: Hey Player_13! My name is PLayer_
14! Its funny how our names are so similar...
>
```

```
[PUBLIC_SYSTEM] [phase 2 · DayOne] Day 1 introductions
 have begun. No trial is held today.
[PUBLIC] #4 Player_13: Hello Everyone! My name is Play
er_13!
> /say Hey Player_13! My name is PLayer_14! Its funny
how our names are so similar...
[PUBLIC] #5 Player_14: Hey Player_13! My name is PLaye
r_14! Its funny how our names are so similar...
[OK] Message delivered.
>
```

```
Scene: Role Reveal (Day 1) -> Day 1 Introductions (Day 1)
[CHAT:PUBLIC] #4 Player_13: Hello Everyone! My name is Player_13!
[CHAT:PUBLIC] #5 Player_14: Hey Player_13! My name is PLayer_14! Its funny how our names are so similar...
```

Players communicating with each other during the day using /say from both players and server POV

```
[PUBLIC_SYSTEM] [phase 3 · Night] Night 1 has begun.
[PRIVATE_SYSTEM] Night commands for your Mafioso role: /kill ID, /cancel, /mafia MES
SAGE, or /deathnote set TEXT. Basic commands: /players to find IDs, /role, /help Maf
ioso, and /will set TEXT.
>
```

Day ending and night starting. Night is where most roles come into play and gives them some kind of action to do

```
[PRIVATE_SYSTEM] You are the Jester (Neutral). Self-heals
: 1. Executions: 3. Role guide: /help Jester
> /will set I am the Jester hehehhee
```

This is a command called /will set. A will in this game is something that will be shown to the lobby on death and can be used to throw off who the mafia is or support it is they were a good person, it adds a lot of complexity to the game and what to trust.

```
> /investigate #2
[PRIVATE_SYSTEM] Night choice set to #2. You may revise it before cutof
f.
[PRIVATE_SYSTEM] Night 1: no suspicious activity found for #2. This doe
s not prove Town alignment.
[PUBLIC_SYSTEM] Nobody died during the night.
>
```

The investigator used his investigation on the doctor. Since the doctor is not suspicious, no activity was found.

```
[PUBLIC_SYSTEM] [phase 3 · Night] Night 1 has begun.
[PRIVATE_SYSTEM] Night commands for your Mafioso role: /kill ID, /cancel, /mafia MES
SAGE, or /deathnote set TEXT. Basic commands: /players to find IDs, /role, /help Maf
ioso, and /will set TEXT.
> /players
[PRIVATE_SYSTEM] #1 Player_10 [alive], #2 Player_11 [alive], #3 Player_12 [alive], #
4 Player_13 [alive], #5 Player_14 [alive]
> /kill 2
[PRIVATE_SYSTEM] Night choice set to #2. You may revise it before cutoff.
[MAFIA_SYSTEM] #4 updated their Mafioso selection.
[PRIVATE_SYSTEM] The attack on #2 failed.
[MAFIA_SYSTEM] #4 was promoted to Godfather for future phases.
[PUBLIC_SYSTEM] Nobody died during the night.
>
```

The mafia tries to kill the doctor, but as you will see it fails. Since he is the only mafia member in this match, it promotes him to Godfather (head of mafia)

```
[PRIVATE_SYSTEM] #1 Player_10 [alive], #2 Player_11 [alive], #3 Player
_12 [alive], #4 Player_13 [alive], #5 Player_14 [alive]
> /heal 2
[PRIVATE_SYSTEM] Night choice set to #2. You may revise it before cuto
ff.
[PRIVATE_SYSTEM] Your protected target #2 was attacked.
[PRIVATE_SYSTEM] You were attacked and protected; the Doctor's identit
y remains hidden.
[PUBLIC_SYSTEM] Nobody died during the night.
>
```

The doctor protects himself and ends up not dying despite being attacked. Even though he did not die, the game does tell him that he was attacked.

```
[PRIVATE_SYSTEM] Night commands for your Lookout role: /watch ID or /cancel. Basic commands: /players to find IDs, /role, /help Lookout, and /
will set TEXT.
> /watch #1
[PRIVATE_SYSTEM] Night choice set to #1. You may revise it before cutoff.
[PRIVATE_SYSTEM] Night 1: visitors to #1: no visitors.
[PUBLIC_SYSTEM] Nobody died during the night.
>
```

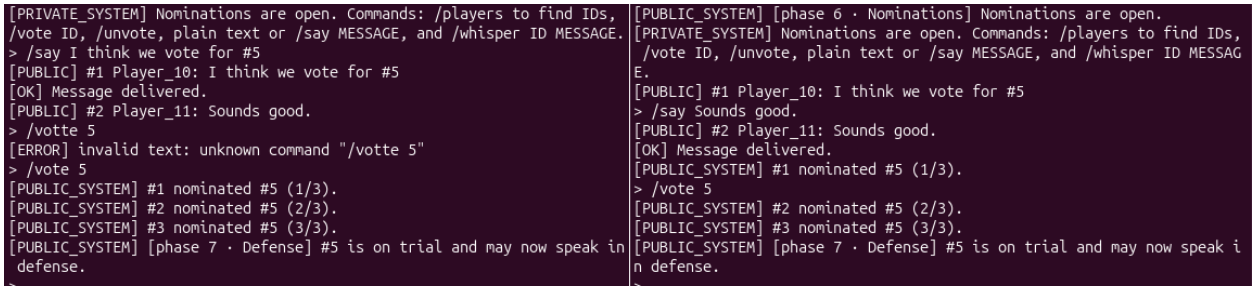
The lookout used his ability /watch to watch the investigator (Player 1). Since player 1 did not leave his house and nobody else visited his house, no visitors were spotted.

```
[PUBLIC_SYSTEM] Nobody died during the night.
> /say I investigated #2 so i think he is good.
[ERROR] command requires an eligible public speaking phase; current pha
se is Dawn
[PUBLIC_SYSTEM] [phase 5 · Discussion] Discussion has begun.
> /say I investigated #2 so i think he is good.
[PUBLIC] #1 Player_10: I investigated #2 so i think he is good.
[OK] Message delivered.
>
```

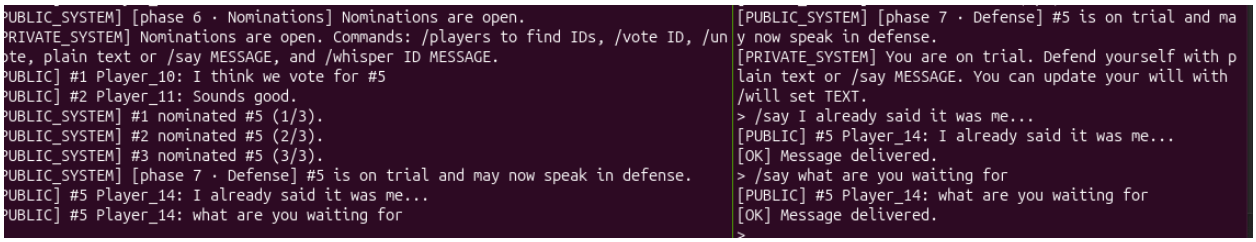
This screenshot shows the player trying to publicly speak during dawn (time between night and the day) and the game not letting him since public speaking is only available during the day (Discussion, Nomination, etc).

```
Scene: Dawn (Day 2) -> Discussion (Day 2)
[CHAT:PUBLIC] #1 Player_10: I investigated #2 so i think he is good.
[CHAT:PUBLIC] #3 Player_12: I watched #1 and nobody visted him. i think i might trust him.
[CHAT:PUBLIC] #2 Player_11: Someone tried to kill me last night!!!!
[CHAT:PUBLIC] #5 Player_14: It was ME who tried to kill the doctor HAHAHHA
```

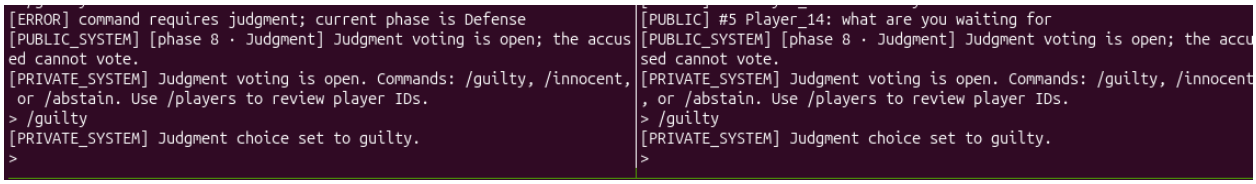
This shows the discussion during the second day where they talked about who they think the mafia may be.



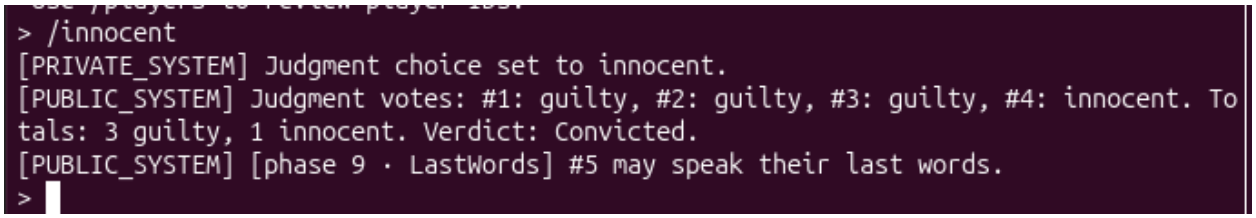

This is a screenshot of the Nomination scene where players can vote for another player to put on trial.



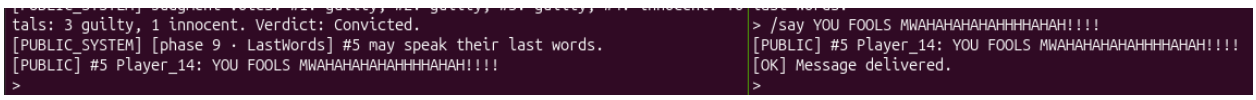

Player 5 was voted so it entered a period where player 5 is able to defend himself.




After the player on trial defends himself, the players get to vote on whether they think they are guilty, innocent, or they can abstain from voting entirely.




The results of the voting shows who voted for what and then tallies them up. In this case, players 1,2, and 3 voted guilty while player 4 voted innocent. Player 5 couldnt vote since they were the one on trial. Because guilty had the majority, he was convicted and given the chance to say his last words.




He uses his last words to call them fools since his role was the joker where his entire objective is to get voted out.

```
[PUBLIC_SYSTEM] #5 Player_14 was publicly executed. Role: Jester. Will: I am the Jes
ter hehehhee
```

After his death and final words, it also prints whatever he wrote in his will into chat for all to read.

```
[PRIVATE_SYSTEM] Night 2: no suspicious activity found for #4. This doe
s not prove Town alignment.
[PUBLIC_SYSTEM] #2 Player_11 died. Causes: {JesterHaunt}. Role: Doctor.
 Will: (empty)
[PUBLIC_SYSTEM] #3 Player_12 died. Causes: {Mafia}. Role: Lookout. Will
: (empty)
```

This screenshot shows the next night. Player_11 dies because the jester chose to haunt him after his  death (part of his role) and Player_!2 died to the mafia member killing him. This is shown to all players.

```
> /vote 4
[PUBLIC_SYSTEM] #1 nominated #4 (1/2).
[PUBLIC_SYSTEM] #4 nominated #1 (1/2).
[PUBLIC_SYSTEM] [phase 14 · Night] Nomination time expired. Night has b
egun.
[PRIVATE_SYSTEM] Night commands for your Sheriff role: /investigate ID
or /cancel. Basic commands: /players to find IDs, /role, /help Sheriff,
 and /will set TEXT.
>
```

This shows a vote where there are equal numbers of votes. 1 for #4 and 1 for #1. Because of this, nobody gets voted and it skips to night.

```
[PUBLIC_SYSTEM] [phase 14 · Night] Nomination time expired. Night has begun.
[PRIVATE_SYSTEM] Night commands for your Godfather role: /kill ID, /cancel, /mafia M
ESSAGE, or /deathnote set TEXT. Basic commands: /players to find IDs, /role, /help G
odfather, and /will set TEXT.
> /kill 1
[PRIVATE_SYSTEM] Night choice set to #1. You may revise it before cutoff.
[MAFIA_SYSTEM] #4 updated their Godfather selection.
[PUBLIC_SYSTEM] #1 Player_10 died. Causes: {Mafia}. Role: Sheriff. Will: (empty)
[PUBLIC_SYSTEM] Match result: MafiaWin.
[PUBLIC_SYSTEM] Final faction result: MafiaWin. #1 Player_10: original=Sheriff, curr
ent=Sheriff, outcome=Loss; #2 Player_11: original=Doctor, current=Doctor, outcome=Lo
ss; #3 Player_12: original=Lookout, current=Lookout, outcome=Loss; #4 Player_13: ori
ginal=Mafioso, current=Godfather, outcome=Win; #5 Player_14: original=Jester, curren
t=Jester, outcome=Win
[PUBLIC_SYSTEM] Postgame: everyone may chat using /say or bare text. Read commands r
emain available; gameplay is frozen. Use /quit to leave.
>
```

Since there are only 2 people left and one is a mafia and the other one can only investigate, the mafia wins by killing the last player. It then reveals everyones roles and lets everyone talk in a post game chat. They can then leave the server by using /quit.




This was just to show the /quit command works.

**Classmate Feedback**

I posted feedback for my game mainly asking if there were any other side channel attacks that people could find to leak player role information and to also test if there are any resource based crashes after implementing cgroup.

Hi everyone!

My project is a game called Fearless Conspiracy which is a rust based game inspired by Town of Salem for 3-9 players. The goal of the game is for Town to try to find the mafia, while the Mafia tries to eliminate the Town (with some shenanigans of course like the jester role).

It uses Unix-domain stream sockets with JSON messages. The communications library is all in src/comm.rs and supports private replies, group chat, and broadcasts. Reader and writer threads handle multiple players at once.

For this project, I focused on isolation. From the previous feedback, i learned that a Town role could watch server threads and look at their activity to figure out who was receiving Mafia messages. This allowed them to deduce who the Mafia was. To stop this, i enabled hidepid = 2 to block access to metadata and then i also added cgroup limits to help possibly contain resource abuse. The game now automatically makes 5 player roles on start and waits for the set amount of players to join based on the parameters passed in. This is all cleaned up on server close, and if the cleanup gets interrupted, there is a recovery option to set back the settings.

Some feedback on

- If there is another way to figure out who mafia is through side channel or any attacks?
- Any methods to still resource abuse crash the game despite the cgroup set up
- Or anything else you would like to talk about!

The code can be found at This GithubLinks to an external site.

For this to work, you need to run it on Ubuntu with cgroup v2. Thank you!

The main feedback that I received was that even though i did stop the /proc route, i forgot to check the /sys/fs/cgroup route made by the new cgroup changes. This allows people to poll other accounts slices and watch for CPU blips which is another side channel attack. Its small and definitely harder to detect leaked information in my opinion, but still a leak. I also need to

worry more about how many connections a single account can open to not block other accounts.

Brinda Udayakumar
Nov 13 6:03pm

Hi Mitchell, I went through your repo and this is really solid, the isolation work clearly went deep. A few notes.

What I think is strong: you don't trust the name a client sends, you use the kernel's SO_PEERCRED to get the real Linux UID behind each connection and reserve it, so I couldn't see a way to impersonate another player the way the original named-pipe style games could be fooled. The display name is just cosmetic, and you validate it (length, charset, reserved names, duplicate check). The frame cap, handshake timeout, per-peer queues, and the rate limiter that disconnects after repeated violations all line up well. Filtering terminal control characters out of chat was a nice touch I didn't think of.

On your side-channel question: hidepid=2 closes the /proc route (a player can't read another player's schedstat to time the night actions), which is the main one. The gap it does not cover is /sys/fs/cgroup. If the per-player user-slice stat files (cpu.stat, memory.current, io.stat) are world-readable, a player could poll another account's slice and watch for a CPU or memory blip when that account acts. In practice the per-action work is tiny and most of it happens in the server process, so the leak may be small, but it's an exposure hidepid doesn't close, so it could be worth tightening the permissions on those cgroup files.

On crashing it despite cgroups: cgroup v2 caps CPU, memory, IO bandwidth, and PIDs, but not disk space or inodes. A player who fills a shared filesystem (their own home, or /tmp) could starve the server's own writes. Also max_incomplete_handshakes looks like a small global limit, so one account could open a few connections that send nothing and sit in those slots for the 5s timeout, which might stall other players from joining during the lobby. A per-UID handshake cap would close that.

Nice work overall, the UID-based identity is the part I'd point other people to.

**Feedback I gave to another student**

For my feedback, I started by playing the game and getting a general vibe for gameplay change suggestions and also looking for possible trouble areas that I could delve deeper into the code for. Doing this, I identified some possible areas like the final voting phase, where I found 2 separate issues that I talked about during the feedback which had to do with disconnected players and also an issue with mole logic when there are more than 4 players.
After that, I wanted to test what would happen if one of the clients was sending a chat but never read from its socket. When doing this test, the game froze up and stalled.

Finally, I wanted to see if there was a limit put on the player ID or if it could be any length I wanted. I figured out it COULD be any length which caused the final message to exceed the size limit.

Mitchell Bialecki
Nov 13 10:34am

Hey Nauman!

I ran your game and looked at the source code and was able to find some possible edge cases or things that may be worth checking.

- Whenever there are five or six players, the game assigns two moles, but extraction only eliminates one player. Since the crew needs every mole gone, they cant win while both moles remain connected, even if they crack the vault and identify one correctly. Adding another elimination round when there is 2 moles would help with this.
- A client is able to stall the game if it sends chats but stops reading responses. It causes the other players to stop receiving vault updates. The game then continued after this client disconnected. A timeout for outgoing messages could help keep a stalled connection holding the game up.
- Late joiners can act without a role, but its drill action still increases vault progress by four points. Closing the lobby after the game starts OR making late joining like spectators instead could prevent this
- Votes are able to be attached to disconnected players. Meaning that a departed player can be selected for elimination. Making it so that they cant be targeted would be a good change and if votes have been cast on someone and they disconnect, those votes should be refunded.
- Finally, if the players have a VERY LONG player ID, the final message will exceed the communication library's size limit. This would just be fixed with a name length limit most likely.

For gameplay, I think giving roles more changes to how they play would be nice. For example, there could be like a hacker role which gets more points for drilling then others, or a lookout role that can receive early warnings, or muscle which can clear objects more efficiently. Something like that.

I also think that giving some more time to talk would add to it, as right now it seems kind of fast but maybe thats just me.

I also think a final discussion phase before the extraction voting would be very nice. It would make giving accusations and defenses more room to have an effect and add on to the social deduction part of the game.

Great job on the game you made! I had fun testing it and poking around it.

I also gave feedback to Brinda where I played her game and once again looked for possible areas where there could be issues, including the areas she specifically stated in her post. I was able to find some workarounds to things like bypassing spam prevention and also locking out real players from joining. I also was able to trick the game as the player into believing a real person left and some fake person joined.

Hi Brinda,

I played your game, looked at your code, and tried some extra attacks i could think of. I could not log in as another player without their key from what i found or read the private messages but I did find some things of interest

- Opening 16 connections and leaving the login questions unanswered would prevent real players from ever joining. This might be able to be fixed by limiting how many unfinished logins one source can hold.
- Reconnection resets the spam limit. Using this i could get 200 chat messages accepted in under two seconds by reconnecting with the same player key. The heartbeat messages also skipped the limit entirely. Keeping the limit tied to the player across reconnects and covering heartbeat traffic too would close these gaps.
- A player can send __join__ and __leave__ messages and the game treat it like actual login changes. These messages should only come from the server itself.
- There is also just weird errors with input such as using the 'é' character or using line breaks or terminal escape characters which sometimes caused crashes or just displaying weird lines.

Overall I had messing around with your game. Good job!

**Closing Reflection**

The first part of this project initially helped me learn a lot about how stream sockets work and how to frame them properly so that they can handle partial reads and disconnects and such. It also taught me how to use concurrent I/O threads in rust to keep communication as responsive as possible.

For this new part of the project, it taught me how to consider what players should and should not have access to, and taught me how to change my thinking to be more adversarial. It also taught

me more about how isolation mechanisms like cgroups work, and how even they have their own side channel attacks that can leak information.
Overall I think I learned a lot about how these attacks can play out over the operating system and gave me a much better understanding of how operating systems in general work.